Snowflake Practice Assessments - Alternate Domains

4 Easy + 3 Medium | Screenshot-Based Trainee Practice Pack - Set B

Easy: 10 Marks / 20 Minutes | Medium: 20 Marks / 45 Minutes

Design note: This alternate set uses completely different business domains from the previous practice pack while retaining the same topic coverage, marks, evidence model and target duration. Each question is intentionally scoped for timed milestone practice.

Assessment	Difficulty	Topic	New Domain	Time / Marks
Easy 1	Easy	Time Travel + Load/Unload + Sharing	Aerospace Spare Parts	20 min / 10
Easy 2	Easy	JSON + LATERAL FLATTEN	Food Delivery	20 min / 10
Easy 3	Easy	Cortex Analyst	Renewable Energy	20 min / 10
Easy 4	Easy	RBAC + Row Access Policy	University Admissions	20 min / 10
Medium 1	Medium	Snowpipe	Agricultural Commodity Markets	45 min / 20
Medium 2	Medium	Snowpark	Airline Ticket Revenue	45 min / 20
Medium 3	Medium	Streamlit in Snowflake	Media Streaming	45 min / 20

Easy 1 - Aerospace Spare Parts: Time Travel, Data Load/Unload & Data Sharing

Difficulty	Easy	**Total Marks**	10
Target Duration	20 minutes	**Dataset**	E1_Aerospace_Spare_Parts.csv

Curriculum Coverage

Snowflake named internal stages, CSV file format, COPY INTO, Time Travel, data unloading, secure views and secure data sharing.

Problem Description

You are working as a Snowflake developer for an aircraft maintenance organization. Spare-parts transaction data must be loaded into Snowflake. A recent incorrect cost update and accidental transaction deletion must be investigated using Time Travel. Approved transactions must then be unloaded for downstream maintenance planning, and a non-sensitive view must be prepared for secure sharing.

Dataset

Dataset: E1_Aerospace_Spare_Parts.csv

Domain: Aerospace Maintenance & Spare Parts

Columns / Structure: TxnID, TxnDate, AircraftModel, PartNumber, PartCategory, SupplierCode, Warehouse, Region, Quantity, UnitCost, TxnStatus

Assessment Requirements

1. Create a Snowflake table matching the dataset, create/use a named internal stage, upload the CSV and load it using COPY INTO. Validate the final row count and display sample records.
2. Perform exactly one UPDATE on UnitCost for a selected transaction and one DELETE on a different transaction. Use Time Travel to retrieve the affected records as they existed before the changes.
3. Unload only APPROVED transactions to a stage in CSV format and verify the generated file(s) using LIST.
4. Create a SECURE VIEW exposing TxnID, TxnDate, AircraftModel, PartNumber, PartCategory, Warehouse, Region, Quantity, UnitCost and TxnStatus while excluding SupplierCode.
5. Create a secure share for the secure view, grant the required database/schema/view privileges and validate the share using SHOW SHARES.

Submission Instructions

Submit the following evidence in serial order. Use consolidated screenshots where possible so that the SQL/code and the corresponding result are visible together.

6. Load validation showing row count and sample records.

7. Time Travel output showing original values before UPDATE/DELETE.
8. Unload command plus LIST output showing staged file(s).
9. Secure view definition/output proving SupplierCode is not exposed.
10. Share creation/grants plus SHOW SHARES output.

Evaluation Rubric

Task	Validation Criterion	Marks
Task 1	Dataset is loaded correctly and row count is validated.	2
Task 2	Time Travel correctly retrieves the historical state of the updated/deleted records.	2
Task 3	APPROVED transactions are unloaded successfully and staged files are visible.	2
Task 4	Secure view returns the required columns and excludes SupplierCode.	2
Task 5	Secure share is created with correct object access and is visible in SHOW SHARES.	2
Total	**Total**	**10**

Evaluation Note: Award marks only when the required Snowflake result is clearly visible in the submitted evidence. SQL/code without execution evidence is insufficient.

Easy 2 - Food Delivery Orders: JSON Access & LATERAL FLATTEN

Difficulty	Easy	**Total Marks**	10
Target Duration	20 minutes	**Dataset**	E2_Food_Delivery_Orders.json

Curriculum Coverage

VARIANT, JSON file format, object/field path notation, nested JSON objects, arrays, LATERAL FLATTEN and structured analytical views.

Problem Description

A food-delivery platform stores each order as a JSON document containing customer, restaurant, payment and ordered-item details. Load the raw JSON, access nested attributes, flatten the items array and create a compact analytical result.

Dataset

Dataset: E2_Food_Delivery_Orders.json

Domain: Food Delivery & Restaurant Orders

Columns / Structure: Newline-delimited JSON. Top level: order_id, order_date, order_status. Nested objects: customer, restaurant, payment. Array: items(item_code, item_name, quantity, unit_price, line_amount).

Assessment Requirements

11. Create a table with a VARIANT column, create/use a JSON file format and stage, and load the JSON dataset. Display sample raw documents.
12. Access nested attributes and display Order ID, Order Date, Customer Name, Customer City, Restaurant Name, Cuisine, Payment Method, Order Amount and Order Status using JSON path expressions.
13. Use LATERAL FLATTEN on the items array and display Order ID, Item Code, Item Name, Quantity, Unit Price and Line Amount as individual rows.
14. Create a view named VW_FOOD_ORDER_ITEMS combining order-level and item-level details with numeric QUANTITY, UNIT_PRICE and LINE_AMOUNT fields.
15. Using the view, display Cuisine, Item Count and Total Item Amount, sorted by Total Item Amount descending.

Submission Instructions

Submit the following evidence in serial order. Use consolidated screenshots where possible so that the SQL/code and the corresponding result are visible together.

16. Raw VARIANT data successfully loaded.

17. Nested JSON attributes extracted correctly.
18. LATERAL FLATTEN output showing one row per ordered item.
19. VW_FOOD_ORDER_ITEMS output.
20. Cuisine-level aggregation output.

Evaluation Rubric

Task	Validation Criterion	Marks
Task 1	JSON is loaded into VARIANT correctly.	2
Task 2	Nested customer/restaurant/payment attributes are accessed correctly.	2
Task 3	LATERAL FLATTEN expands the items array correctly.	2
Task 4	Structured view contains the required order/item fields with usable numeric values.	2
Task 5	Aggregation produces correct cuisine-level totals and counts.	2
Total	**Total**	**10**

Evaluation Note: Award marks only when the required Snowflake result is clearly visible in the submitted evidence. SQL/code without execution evidence is insufficient.

Easy 3 - Renewable Energy Performance using Cortex Analyst

Difficulty	Easy	**Total Marks**	10
Target Duration	20 minutes	**Dataset**	E3_Renewable_Energy_Generation.csv

Curriculum Coverage

Cortex Analyst, semantic model/semantic view, dimensions, measures, natural-language questions, generated SQL and result validation.

Problem Description

A renewable-energy operator wants analysts to ask natural-language questions about plant generation and revenue. Configure a compact semantic layer on the supplied generation table, ask two business questions in Cortex Analyst and validate one answer independently in SQL.

Dataset

Dataset: E3_Renewable_Energy_Generation.csv

Domain: Renewable Energy Generation

Columns / Structure: GenerationID, GenerationDate, PlantName, PlantType, State, Region, CapacityMW, EnergyMWh, EfficiencyPct, DowntimeHours, RevenueAmount

Assessment Requirements

21. Load the generation dataset into a Snowflake table and validate the row count plus sample records.
22. Create/configure a semantic model or semantic view with dimensions PlantName, PlantType, State, Region and GenerationDate; and measures Total Energy, Average Efficiency and Total Revenue.
23. Ask Cortex Analyst: "What is the total energy generated by region?" and display the response.
24. Ask Cortex Analyst: "Which plant type has the highest average efficiency?" and display the generated SQL together with the result.
25. Run an independent SQL query for total EnergyMWh by Region and verify that the result agrees with Cortex Analyst.

Submission Instructions

Submit the following evidence in serial order. Use consolidated screenshots where possible so that the SQL/code and the corresponding result are visible together.

26. Dataset validation output.
27. Semantic configuration showing the required dimensions and measures.
28. First Cortex Analyst response for regional energy.
29. Second Cortex Analyst response with generated SQL.

30. Independent SQL validation result.

Evaluation Rubric

Task	Validation Criterion	Marks
Task 1	Dataset is loaded and validated correctly.	2
Task 2	Semantic configuration contains the required business dimensions and measures.	2
Task 3	Cortex Analyst correctly answers regional total energy.	2
Task 4	Cortex Analyst returns a valid plant-type efficiency answer and generated SQL.	2
Task 5	Independent SQL confirms the regional Cortex Analyst result.	2
Total	**Total**	**10**

Evaluation Note: Award marks only when the required Snowflake result is clearly visible in the submitted evidence. SQL/code without execution evidence is insufficient.

Easy 4 - University Admissions Security using RBAC & Row Access Policy

Difficulty	Easy	**Total Marks**	10
Target Duration	20 minutes	**Dataset**	E4_University_Admissions.csv

Curriculum Coverage

RBAC, role creation, privilege grants, role-to-region mapping, Row Access Policy, policy application and access validation.

Problem Description

A university group operates campuses across multiple regions. Regional admissions teams must see only applications belonging to their assigned region, while the central admissions administrator must see every region. Implement this requirement using Snowflake RBAC and a Row Access Policy.

Dataset

Dataset: E4_University_Admissions.csv

Domain: Higher Education & Admissions

Columns / Structure: ApplicationID, ApplicationDate, ApplicantID, ApplicantName, Program, Campus, Region, EntranceScore, ApplicationStatus, ScholarshipAmount

Assessment Requirements

31. Load the admissions dataset into a Snowflake table and validate that NORTH, SOUTH, EAST and WEST data are present.
32. Create roles ADMISSIONS_NORTH, ADMISSIONS_SOUTH and ADMISSIONS_ADMIN. Grant only the privileges required to query the assessment objects.
33. Create a role-to-region mapping table and a Row Access Policy that allows regional roles to see only their mapped region while ADMISSIONS_ADMIN can see all rows. Apply the policy to Region.
34. Switch to ADMISSIONS_NORTH and validate that only NORTH applications are returned. Then switch to ADMISSIONS_SOUTH and validate that only SOUTH applications are returned.
35. Switch to ADMISSIONS_ADMIN and validate that applications from all available regions are visible.

Submission Instructions

Submit the following evidence in serial order. Use consolidated screenshots where possible so that the SQL/code and the corresponding result are visible together.

36. Source table validation showing multiple regions.
37. Role creation and privilege grants.
38. Mapping table plus Row Access Policy definition/application.

39. Regional-role validation outputs.
40. ADMISSIONS_ADMIN output showing all regions.

Evaluation Rubric

Task	Validation Criterion	Marks
Task 1	Admissions data is loaded and multi-region data is confirmed.	2
Task 2	Required roles and privileges are created correctly.	2
Task 3	Role mapping and RAP logic are correctly created and applied.	2
Task 4	Regional roles can see only their assigned rows.	2
Task 5	ADMISSIONS_ADMIN can see rows across all regions.	2
Total	**Total**	**10**

Evaluation Note: Award marks only when the required Snowflake result is clearly visible in the submitted evidence. SQL/code without execution evidence is insufficient.

Medium 1 - Snowpipe Auto-Ingest for Agricultural Commodity Price Feeds

Difficulty	Medium	**Total Marks**	20
Target Duration	45 minutes	**Dataset**	M1_Snowpipe_Commodity_Batches.zip (3 CSV files)

Curriculum Coverage

Snowpipe, external stages, AUTO_INGEST, continuous/incremental loading, SYSTEM$PIPE_STATUS, COPY_HISTORY/load history and validation.

Problem Description

An agricultural market analytics platform receives commodity price files throughout the day from regional market feeds. Configure Snowpipe to ingest each arriving batch automatically and prove that the pipeline processes incremental files without duplicate or missing PriceEventID values.

Dataset

Dataset: M1_Snowpipe_Commodity_Batches.zip (3 CSV files)

Domain: Agricultural Commodity Markets

Columns / Structure: Files: commodity_price_batch_01.csv, commodity_price_batch_02.csv, commodity_price_batch_03.csv. Columns: PriceEventID, EventTimestamp, MarketCode, State, Region, Commodity, Grade, ModalPricePerQuintal, ArrivalTons, SourceSystem.

Timed-practice prerequisite: Trainer provides an accessible cloud object-storage location, storage integration/credentials and event-notification setup so candidates focus on Snowpipe rather than cloud administration.

Assessment Requirements

41. Create the target COMMODITY_PRICE_EVENTS table, CSV file format and external stage for the trainer-provided cloud location. Confirm that batch 01 is visible in the stage.
42. Create a Snowpipe with AUTO_INGEST = TRUE that loads staged commodity-price files into COMMODITY_PRICE_EVENTS. Display the pipe definition and status.
43. Place batch 01 in the source location and validate the initial load using target row count and sample records.
44. Place batches 02 and 03 at different times. Record the row count after each arrival and confirm that all PriceEventID values remain unique.
45. Review pipe status and load history. Then show record count and average ModalPricePerQuintal grouped by Commodity, and confirm that none of the three provided files failed to load.

Submission Instructions

Submit the following evidence in serial order. Use consolidated screenshots where possible so that the SQL/code and the corresponding result are visible together.

46. Stage/table/file-format validation.
47. Snowpipe creation plus pipe definition/status.
48. Initial batch load result and row count.
49. Incremental row-count evidence after batches 02 and 03.
50. Load history/status plus Commodity summary.

Evaluation Rubric

Task	Validation Criterion	Marks
Task 1	Target table, file format and external stage are configured correctly.	4
Task 2	Snowpipe is correctly created for automated ingestion.	4
Task 3	Initial batch loads successfully with correct records.	4
Task 4	Incremental batches load without loss or duplicate PriceEventID values.	4
Task 5	Pipe status/load history and final commodity summary confirm successful ingestion.	4
Total	**Total**	**20**

Evaluation Note: Award marks only when the required Snowflake result is clearly visible in the submitted evidence. SQL/code without execution evidence is insufficient.

Medium 2 - Airline Revenue Processing using Snowpark Python

Difficulty	Medium	**Total Marks**	20
Target Duration	45 minutes	**Dataset**	M2_Airline_Ticket_Revenue.csv

Curriculum Coverage

Snowpark Python DataFrames, schema inspection, duplicate/null handling, expressions, filtering, aggregations, save_as_table and analytical validation.

Problem Description

An airline requires a Snowpark Python workflow to clean ticket-booking data and generate route-level revenue insights. The source intentionally contains duplicate TicketID rows and null FareAmount values so that data-quality handling can be demonstrated.

Dataset

Dataset: M2_Airline_Ticket_Revenue.csv

Domain: Airline Ticketing & Revenue

Columns / Structure: TicketID, BookingDate, PassengerID, Origin, Destination, CabinClass, BookingChannel, FareAmount, TaxAmount, TicketStatus, DelayMinutes. Data quality: 3 duplicate rows and 3 records with null FareAmount.

Assessment Requirements

51. Using Snowpark Python, read AIRLINE_TICKETS into a DataFrame and display schema, source row count and sample records.
52. Clean the DataFrame by removing duplicate TicketID rows, excluding null FareAmount records and retaining only TicketStatus = CONFIRMED. Display source count, clean count and removed count.
53. Create FARE_BAND using Snowpark expressions: BUDGET < 5,000; STANDARD 5,000-9,999.99; PREMIUM >= 10,000. Display a representative sample.
54. Create ROUTE as Origin + "-" + Destination and aggregate by ROUTE to calculate Ticket Count, Total Fare, Average Fare and Maximum Fare. Sort by Total Fare descending and save as ROUTE_REVENUE_SUMMARY.
55. Filter PREMIUM fare-band records and aggregate by CabinClass and BookingChannel to produce PREMIUM_TICKET_COUNT and PREMIUM_FARE_TOTAL. Display the results ordered by total fare descending.

Submission Instructions

Submit the following evidence in serial order. Use consolidated screenshots where possible so that the SQL/code and the corresponding result are visible together.

56. Snowpark schema/count/sample output.
57. Cleaning validation with source, clean and removed counts.
58. FARE_BAND output.
59. Route summary output plus persisted-table validation.
60. Premium fare analysis by CabinClass and BookingChannel.

Evaluation Rubric

Task	Validation Criterion	Marks
Task 1	Snowpark DataFrame correctly accesses and inspects the source table.	4
Task 2	Duplicate, null and status filtering rules are implemented correctly.	4
Task 3	FARE_BAND classification is implemented correctly using Snowpark expressions.	4
Task 4	Route-level metrics are calculated correctly and persisted to ROUTE_REVENUE_SUMMARY.	4
Task 5	Premium-ticket analysis is correctly filtered, aggregated and ordered.	4
Total	**Total**	**20**

Evaluation Note: Award marks only when the required Snowflake result is clearly visible in the submitted evidence. SQL/code without execution evidence is insufficient.

Medium 3 - Media Streaming Performance Dashboard using Streamlit in Snowflake

Difficulty	Medium	**Total Marks**	20
Target Duration	45 minutes	**Dataset**	M3_Media_Streaming_Performance.csv

Curriculum Coverage

Streamlit in Snowflake, active Snowpark session, KPI metrics, interactive filters, charts, detail tables and business analysis.

Problem Description

A streaming-media platform wants a compact single-page dashboard for content consumption. Build a Streamlit in Snowflake application that reads the supplied session data, responds to filters and summarizes engagement and monetization metrics.

Dataset

Dataset: M3_Media_Streaming_Performance.csv

Domain: Media Streaming & OTT Analytics

Columns / Structure: SessionID, SessionDate, UserID, ContentTitle, Genre, Country, SubscriptionTier, DeviceType, WatchMinutes, CompletionPct, AdRevenue, SessionStatus

Assessment Requirements

61. Load the dataset into a Snowflake table and create a Streamlit in Snowflake app titled "Streaming Content Performance Dashboard". Connect the app to the table using the active Snowpark session.
62. Display three KPI metrics that update from the data: Total Sessions, Total Watch Hours and Average Completion %. Calculate Total Watch Hours from WatchMinutes.
63. Add two interactive filters: Genre and Country. Ensure selections update the KPI values and downstream displays.
64. Create two different visualizations: Watch Hours by Genre and Monthly Session Trend. Use appropriate chart types and readable labels.
65. Add a filtered detail table and a Top 5 Content Titles by Watch Hours summary. Demonstrate that both respond to the selected filters.

Submission Instructions

Submit the following evidence in serial order. Use consolidated screenshots where possible so that the SQL/code and the corresponding result are visible together.

66. Running Streamlit app with title and connected data.
67. Three KPI metrics.
68. Filter controls with visibly changed results.

69. Two required visualizations.
70. Filtered detail table plus Top 5 Content Titles summary.

Evaluation Rubric

Task	Validation Criterion	Marks
Task 1	Streamlit application is created and connected to Snowflake data.	4
Task 2	All three KPI calculations are correct and data-driven.	4
Task 3	Genre and Country filters are implemented and functional.	4
Task 4	Required visualizations accurately represent filtered data.	4
Task 5	Detail table and Top 5 content analysis are correct and filter-responsive.	4
Total	**Total**	**20**

Evaluation Note: Award marks only when the required Snowflake result is clearly visible in the submitted evidence. SQL/code without execution evidence is insufficient.

Practice Pack Summary

This Set B intentionally uses new domains compared with the previous practice pack: Aerospace, Food Delivery, Renewable Energy, Higher Education, Agricultural Markets, Aviation and Media Streaming.

Exam-plan fit: selecting any 2 Easy + any 2 Medium gives 60 marks. At 20 + 20 + 45 + 45 minutes, the implementation target is 130 minutes, leaving approximately 5 minutes within a 135-minute window for final screenshot review and submission.